AI loop engineering: from zero to enterprise

Course syllabus · version 3 · approved 2026-10-27

Who it's for: Developers who write Python and call a web API; they build, ship and run agent loops and must pass developer interviews. No machine-learning background assumed.

What you'll be able to do: Build an agent loop that stops on a checkable goal, retries safely, asks a human before risky writes, runs unattended on a schedule, survives crashes, and is traced, budgeted and governed like any production service.

33 lessons · about 13 hours

Interview questions & projects: Basic projects (projects/BASIC_PROJECTS.pdf) · Intermediate projects (projects/INTERMEDIATE_PROJECTS.pdf) · Advanced projects (projects/ADVANCED_PROJECTS.pdf)

Tools you'll use

Tool	What for	Lessons	How you can run it
Python 3.12	language for every lesson and the app	01-30	local: Python 3.12 (default)
Gemini API (google-genai)	default model for every loop lesson	02-30	Gemini API key (default) / Vertex AI project and location
Anthropic SDK (anthropic 2.3.0)	optional Claude model for the same lessons	02-30	Anthropic API key (optional)
pydantic	tool argument schemas and validation	04, 05	local: pydantic 2.14.1
tenacity	retries with backoff and jitter	13	local: tenacity 9.2.1
pytest	deterministic verifier and CI eval gate	07, 08, 17, 18	local: pytest 9.1.1
LangGraph	checkpointer and interrupts for durable, approval-gated loops	22, 26	local: langgraph 1.2.17 with the SQLite checkpointer
SQLite (standard library)	loop state, runs and the approval queue	12, 22, 26	local: sqlite3 (standard library)
git and git worktree	state and isolation per run	11, 12, 21	local: git (installed)
FastAPI and uvicorn	the loop as a service	28, 30	local: fastapi 0.162.0, uvicorn 0.54.0
OpenTelemetry SDK	traces of each loop run	27, 28	local: opentelemetry-sdk 1.45.1, console exporter
Arize Phoenix (local)	trace viewer for OpenTelemetry spans	28	local: Phoenix in Docker Compose
Docker Compose	the worker fleet and the trace viewer	28	local: Docker and Compose v2 or later
Claude Agent SDK (claude-agent-sdk 0.2.174)	hooks, max_turns and max_budget_usd in the walkthroughs	10, 16	Anthropic API key (optional, for the walkthroughs)

You'll be asked which option you want, and for any keys, just before the lessons that need it.

Before you start

• Docker installed (some lessons start services in containers)
Put the values in this course's .env file (template: .env.example).

Contents

Basic — 8 lessons · ~3 h

Chapter 1. Foundations: the model call and the loop by hand

01 Why loop engineering, and what a loop is (and is not) · theory — Say what loop engineering adds to prompting and harnesses, and draw the inner and outer loop of a real agent.

02 Your first model call: messages, usage, cost and streaming — Make one model call through the course helper get_llm(), read usage and cost, and stream the answer. get_llm() sets temperature=0 for you, so this lesson never sets temperature, top_p or top_k: the newest Claude models reject them.

03 The agent loop by hand: tool calls, results and stop reasons — Write the model-tool-observe cycle yourself so every later lesson makes sense.

04 Tools the model can call safely: schemas, validation, errors as observations — Validate tool arguments with pydantic and return failures to the model as observations it can recover from.

05 Stop reasons and budgets: caps on turns, tokens, dollars and time — Stop a loop on every exit it can take, and say why caps are checked after a response.

06 Knowing when the loop is stuck: repeated calls and the four exits — Detect a loop that repeats itself, and choose between done, stuck, over budget and hand off.

07 Verifiers: a test is the stop signal, and a golden set checks the loop — Use a mechanical verifier (pytest exit code) as the stop signal and a golden set to measure the loop.

08 Checkpoint: the desk loop with caps, a verifier and a stuck detector ★ — Run the desk loop and the pocket-cart fix loop by hand, each with a mechanical stop and a budget.

Intermediate — 10 lessons · ~4 h

Chapter 2. Control: outer loops, state, retries and review

09 Why one prompt is not enough: outer loops and fresh context per iteration · theory — Explain why real work needs an outer loop, and the four loop types Anthropic names.

10 Goal-based loops: a script Stop hook as the mechanical gate · walkthrough — Gate the end of a loop with a script Stop hook that runs the verifier, and say why a model-judged goal (like /goal) is weaker. Also the Agent SDK budgets: max_turns and max_budget_usd (unlimited by default), with the Anthropic key only for the live run.

11 The Ralph loop: one item per iteration, fresh context, commit on green — Run an outer loop that fixes one failing test per iteration and reverts when the check goes red.

12 State outside the context: progress file, feature list, git and SQLite — Keep loop state in files and a SQLite table so a fresh iteration knows where it is.

13 Retries, backoff and idempotent writes — Retry transient errors with backoff and jitter, and make a refund write safe to repeat.

14 A separate checker: maker and checker split, and reward hacking — Split a loop into a maker and a checker with fresh context, and catch an agent that edits the test instead of the code.

15 Context growth: caching, clearing tool results and compaction — Measure how a long loop's context grows and keep it bounded.

16 Tools, MCP, skills and hooks as the loop's controls · walkthrough — Choose which capability a loop gets: a tool, an MCP server, a skill file or a hook.

17 Evals in CI: golden sets, trajectory checks and pass^k — Run the golden set in CI on recorded model calls, and fail the build when reliability drops.

18 Checkpoint: a Ralph loop with a checker and a CI eval gate ★ — Run the outer loop on pocket-cart with a maker/checker split and a golden-set gate that blocks a bad change, started by

one run command.

Advanced — 15 lessons · ~6 h

Chapter 3. Production: triggers, approvals and safety for unattended loops

19 Why loops run unattended, and what goes wrong · theory — Name the failure modes of an unattended loop, and decide where a human must stay the verifier.

20 Triggers: cron, events and heartbeats with leases — Start a loop on a schedule or an event, and stop two runs from taking the same task.

21 Isolation: one git worktree per run, subagents and review bandwidth — Run several loop iterations in parallel without collisions, and keep the review queue short.

22 Human approval as a loop exit: pause, show arguments, resume — Pause a loop before a refund, show the real arguments, and resume or escalate on the reviewer's answer.

23 Injection and permissions: least privilege for unattended agents — Show how a tool output can take over a loop, and limit each run to the tools it needs.

24 Checkpoint: a triggered loop with approvals and isolated runs ★ — Run a scheduled pocket-cart loop in separate worktrees, with an approval gate and a permission list.

Chapter 4. Enterprise: durable runs, cost, operation and governance

25 Why one loop stops being enough: multi-agent cost, durability and scale · theory — Decide between one loop and several, from the token multiplier and the failure modes.

26a Durable resume: checkpoints, replay and the crash drill — Resume a loop after a crash from its last checkpoint without repeating a side effect, and prove it with a drill.

26b Kill switch and lease expiry: stopping a runaway fleet — Stop every run from one switch, and expire leases so a dead worker's task is taken over once.

27 Observability and cost: traces, loop metrics and per-run budgets — Trace one loop run as a span tree (one span per iteration, attribute loop.iteration), and compute cost per success.

28a A loop as a service: a FastAPI run endpoint and an approval queue — Serve the loop from FastAPI with a run endpoint, a queue and an approval endpoint.

28b A worker fleet in Docker Compose: scale, lease expiry and the drill — Run docker compose up --scale worker=3, expire a worker's lease, and show which limit breaks first (the provider's requests per minute).

29a Tenancy and budgets: per-tenant limits, per-run traps and gateway keys · walkthrough — Set per-tenant budgets and per-run caps, and avoid the trap that a per-run budget is not a ceiling.

29b Identity, audit and rollout: agent identity, shadow runs, canary and rollback · walkthrough — Give each agent an identity, write an audit trail, and roll a model or prompt change out as shadow, then 5% canary, then rollback.

30 Capstone: the enterprise loop, end to end ★ — Build the desk loop as a service with approvals, durable runs, an eval gate and traces, and run it end to end. Acceptance: the kill drill passes; the overshoot is shown and capped; an injected refund instruction is blocked; the eval gate fails a planted bad change.

★ = project lesson

Where next

Harness engineering · discipline · growing

Building the execution environment around a model: tools, guardrails, memory, model routing, evals. Why next: A loop is the control layer above a harness; once loops work you own the environment they run in.

• tool and sandbox design
• context assembly
• guardrails and permissions
• regression evals and failure taxonomies
Start it: /tutorial "Harness engineering" basic reuse:ai-loop-engineering-15h

Agent evaluation and verification platforms · technology · growing

Eval suites, verifiers and release gates for agents at scale. Why next: The verifier is the product of a loop; platforms industrialise it.

• golden sets and trajectory scoring
• LLM-as-judge calibration
• eval gates in CI
• online monitoring
Start it: /tutorial "Agent evaluation and verification platforms" basic reuse:ai-loop-engineering-15h

Context engineering · discipline · growing

Deciding what enters the context each turn: summaries, retrieval, skills, compaction. Why next: Long loops fail on context before they fail on logic.

• compaction and fresh-context iterations
• skills and progressive disclosure
• memory stores
• prompt caching
Start it: /tutorial "Context engineering" basic reuse:ai-loop-engineering-15h

AI platform and agent infrastructure engineer · role · established

Runs the gateway, identity, durable runtime, quotas and observability that many teams share. Why next: Org-wide loops need shared guardrails, budgets and audit.

• agent gateways and identity
• durable execution
• multi-tenant quotas
• audit and governance
Start it: /tutorial "AI platform and agent infrastructure engineer" basic reuse:ai-loop-engineering-15h

Forward deployed engineer · role · established

Embeds with customers to scope, build and run agent solutions on their data. Why next: Loops with verifiers need customer-specific acceptance tests.

• customer scoping
• evals on customer data
• secure integration
• deployment reviews
Start it: /tutorial "Forward deployed engineer" basic reuse:ai-loop-engineering-15h

Agentic software engineering (loop engineering as a practice) · discipline · emerging

Directing coding agents through automated loops; the term dates from June 2026. Why next: This is the course topic taken to team scale.

• maker/checker reviews
• comprehension debt management
• loop metrics
• experiment design
Start it: /tutorial "Agentic software engineering (loop engineering as a practice)" basic reuse:ai-loop-engineering-15h

Appendix: details for reviewers

How this syllabus was checked, and other ways to take it. Learners can skip this part.
Running example — Brightside Outfitters support desk and the pocket-cart repo: a refund makes approvals and idempotency real; a failing test suite gives a loop a mechanical stop check; both need no download. desk loop with caps (chapter 1) -> retries and a checker (chapter 2) -> approvals, triggers and isolated runs (chapter 3) -> durable, traced fleet with tenants (chapter 4).
LLM use: every loop calls a real model; each call is recorded once with a key and replayed without one; at most 5 calls per lesson Recorded once and replayed (no key needed by learners); at most 5 model calls per lesson.

Shorter paths through the same course

Path	Lessons	Time
Essentials	31	~13 h
Professional	33	~13 h
Complete	33	~13 h

Coverage report

37 concepts from 26 independent sources. Must-cover: 31; taught: 30; left out on purpose: 1.

Developer coverage (D1–D18)

Item	Lessons
D1 Purpose and architecture	01, 30
D2 Mental model and core concepts	01, 02, 03
D3 Setup and first working app	02, 08
D4 Day-to-day API surface	02, 03, 04
D5 Data, schemas and configuration	04, 12
D6 Integration with the app stack	16, 28a
D7 Async, concurrency and performance basics	21, 28b
D8 Errors, validation and reliability	04, 13
D9 Testing and evaluation	07, 17
D10 Debugging and observability	06, 27
D11 Security	16, 22, 23
D12 Packaging, deployment and operations	18, 26b, 28b
D13 Scaling and cost	27, 28b
D14 Versions, upgrades and migration	02, 29b
D15 Ecosystem and choosing	16, 25
D16 Common pitfalls and anti-patterns	11, 14
D17 Reference architectures and real use cases	28a, 29b
D18 End-to-end capstone	30

Enterprise depth coverage

Item	Lessons
1 distributed architecture internals	26a
2 cluster operations	28b
3 failure and recovery	26a, 26b
4 extreme scale	28b
5 deployment options	28a, 28b
6 performance engineering at scale	27

Item	Lessons
7 high availability and disaster recovery	26a
8 zero-downtime change	29b
9 multi-tenancy and isolation	29a
10 security and compliance at scale	23
11 cost engineering	27
12 platform and organisation patterns	29b
13 frontier	29b
14 case studies	29b

Out of scope

Kubernetes labs — its own course; the fleet runs on Docker Compose (no Kubernetes labs here)
Fine-tuning and RL for tool use — research level; papers go to Go deeper

Syllabus history

Version	Date	Changes
1	2026-10-27	approved with an earlier kit version
2	2026-10-27	Vendor SDK imports no longer required (course uses the get_llm() helper); lesson 02 goal names get_llm()
3	2026-10-27	Lesson 02 goal: get_llm() sets temperature=0 (checked in llm_setup.py); the lesson never sets it